
Proyecto de Facturacion electronica

20180967 – Angel Moises Lopez Yus

Resumen

El presente proyecto implementa un sistema integral de facturación para servicios de infraestructura en la nube desarrollado para Tecnologías Chapinas S.A. El sistema utiliza una arquitectura cliente-servidor basada en API REST, implementando el paradigma de programación orientada a objetos mediante Python. El backend desarrollado en Flask gestiona recursos tecnológicos (hardware y software), categorías de configuración, clientes, instancias de servicio y consumos, almacenando la información en formato XML como base de datos persistente. El frontend construido con Django proporciona una interfaz web intuitiva que permite la carga de archivos XML, consulta de datos, creación manual de registros y generación automática de facturas. Se implementaron expresiones regulares para validar formatos de datos como NIT y fechas, extrayendo información de textos con formato flexible. El sistema genera reportes en PDF con detalle de facturas y análisis de ventas. La solución cumple con los requerimientos académicos de utilizar POO, persistencia en XML, procesamiento de archivos estructurados y generación de documentación técnica, demostrando la aplicación práctica de tecnologías web modernas en la gestión empresarial de servicios cloud

Palabras clave

POO, API, Facturacion, Rest, XML

Abstract

This project implements a comprehensive billing system for cloud infrastructure services developed for Tecnologías Chapinas S.A. The system uses a client-server architecture based on REST API, implementing the object-oriented programming paradigm using Python. The backend developed in Flask manages technological resources (hardware and software), configuration categories, clients, service instances and consumption, storing information in XML format as a persistent database. The frontend built with Django provides an intuitive web interface that allows XML file upload, data queries, manual record creation and automatic invoice generation. Regular expressions were implemented to validate data formats such as NIT and dates, extracting information from texts with flexible format. The system generates PDF reports with invoice details and sales analysis. The solution meets academic requirements of using OOP, XML persistence, structured file processing and technical documentation generation, demonstrating the practical application of modern web technologies in business management of cloud services.

Keywords

POO Rest, API, XML, Facturation.

Introducción

La computación en la nube ha transformado la manera en que las empresas gestionan su infraestructura tecnológica, permitiendo el aprovisionamiento dinámico de recursos bajo demanda. Este cambio paradigmático requiere sistemas de facturación sofisticados capaces de contabilizar el uso variable de recursos y generar cobros precisos basados en consumo real.

Tecnologías Chapinas S.A., empresa proveedora de servicios de infraestructura en la nube, identificó la necesidad de desarrollar un sistema automatizado que gestione la facturación detallada de sus servicios. El sistema debe manejar múltiples tipos de recursos (hardware y software), configuraciones personalizadas, diversos clientes con múltiples instancias activas, y registros de consumo en tiempo variable.

El presente documento describe el desarrollo e implementación de una solución tecnológica integral utilizando programación orientada a objetos, servicios web RESTful, y persistencia de datos en XML, cumpliendo con los requerimientos establecidos para la gestión eficiente del modelo de negocio de servicios cloud.

Desarrollo del tema

Tecnologías Chapinas S.A. ofrece servicios de infraestructura en la nube organizados en categorías según el tipo de carga de trabajo (desarrollo, producción, testing). Cada categoría contiene múltiples configuraciones que agrupan recursos tecnológicos específicos. Los clientes pueden aprovisionar instancias de estas configuraciones y el

sistema debe registrar el tiempo de uso para calcular los costos correspondientes.

El desafío principal radica en:

- Gestionar jerarquías complejas de datos (categorías → configuraciones → recursos)
- Procesar archivos XML con formatos flexibles
- Validar información mediante expresiones regulares
- Calcular costos basados en múltiples variables ($\text{tiempo} \times \text{cantidad} \times \text{tarifa}$)
- Generar facturas automáticas agrupando consumos por cliente
- Mantener persistencia de datos sin usar bases de datos relacionales.

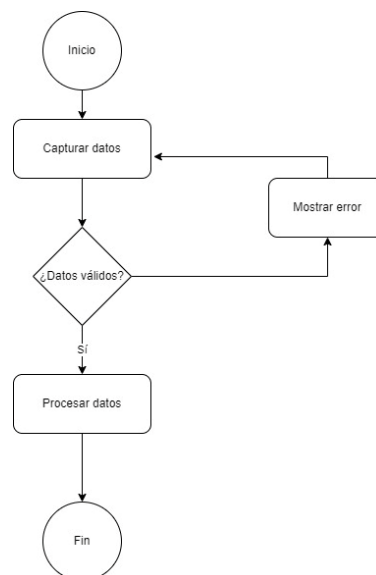


Figura 1. Diagrama de flujo.

Fuente: elaboración propia.

Diseño de la Arquitectura

Se optó por una arquitectura de microservicios con dos componentes principales:

Backend (Flask):

Responsabilidad única: lógica de negocio y gestión de datos

API RESTful con 20+ endpoints

Procesamiento de XML mediante parseo programático

Validaciones con expresiones regulares

Persistencia en archivo XML estructurado
Puerto: 5000

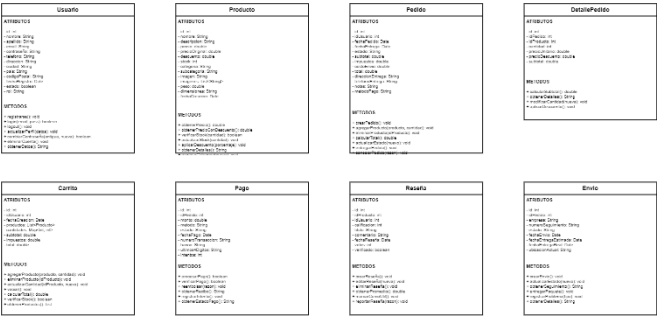


Figura 2. Diagrama de clases.

Fuente: elaboración propia.

Frontend (Django):
Patrón MVT (Model-View-Template)
Interfaz de usuario web responsive
Consumidor de servicios REST
Generación de PDFs con ReportLab
Sistema de autenticación con sesiones
Puerto: 8000

Esta separación permite:
Escalabilidad independiente de componentes
Reutilización del backend por otros clientes
Mantenimiento simplificado
Testing aislado de capas

Implementación de Programación Orientada a Objetos
Se diseñaron siete clases principales siguiendo principios SOLID.

Conclusiones

La implementación de clases con responsabilidades bien definidas facilitó el mantenimiento del código y permitió la reutilización de componentes. El uso de herencia, encapsulamiento y polimorfismo demostró ser fundamental para organizar la lógica de negocio de manera eficiente. La separación entre backend (Flask) y frontend (Django) mediante API REST proporcionó

flexibilidad y escalabilidad. Esta arquitectura permite que el backend sea consumido por múltiples clientes (web, móvil, desktop) sin modificaciones. Aunque XML es menos eficiente que bases de datos relacionales para grandes volúmenes, cumplió satisfactoriamente los requerimientos del proyecto. La ventaja principal es la portabilidad y legibilidad del archivo de datos, facilitando depuración y auditoría.

El proyecto permitió aplicar conocimientos de programación web full-stack, integración de sistemas, procesamiento de datos estructurados, y generación de documentación técnica, habilidades fundamentales para el desarrollo de software empresarial.

Referencias bibliográficas

Python Software Foundation. (2024). Python 3.13 Documentation. Recuperado de <https://docs.python.org/3/>

